MCT-454L Mobile Robotics

Lab 5: Introduction to Gazebo and RViz with ROS 2 using TurtleBot3

1. Objective

The objective of this lab session was to familiarize myself with the Gazebo simulation environment and the RViz visualization tool in ROS 2 using the TurtleBot3 platform. The primary goals included visualizing sensor data, navigating the robot to generate a map using the Cartographer package, and writing custom Python nodes to interact with velocity and odometry topics.

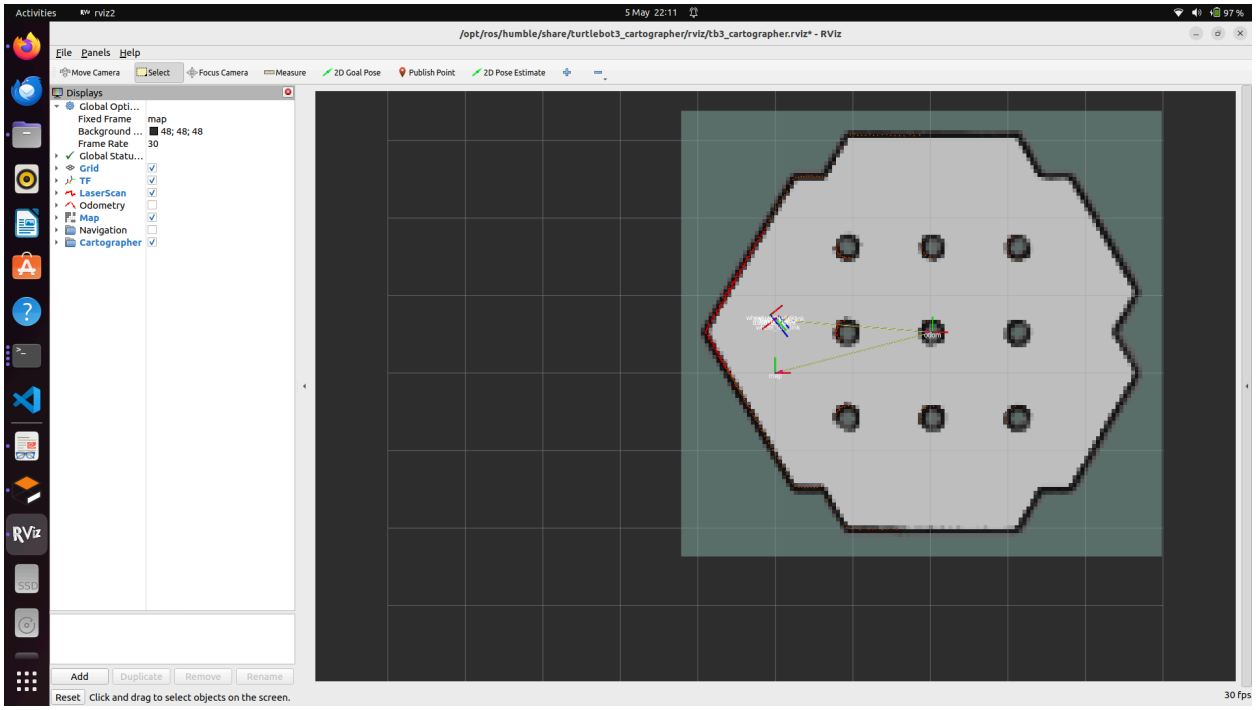
2. Methodology & Steps Followed To achieve the lab objectives, the following sequence of operations was executed:

- **Environment Setup:** The TurtleBot3 model was set to burger and the ROS 2 Humble workspace was sourced.
- **Simulation Launch:** The 3D physics environment was initiated using turtlebot3_gazebo in an empty world.
- **Mapping & Visualization:** RViz and Google's Cartographer were launched simultaneously to visualize the Odometry, TF, Path, and LaserScan data, and to build a SLAM-generated map.
- **Teleoperation & Recording:** The turtlebot3_teleop keyboard node was used to navigate the robot manually. Concurrently, all topic data was recorded using ros2 bag record -a, and the final map was saved via the nav2_map_server.
- **Custom ROS 2 Nodes:** A dedicated ROS 2 package (lab5_turtlebot3_pkg) was created to host two custom Python scripts: a velocity publisher (/cmd_vel) and an odometry subscriber (/odom).

3. Results and Visualizations

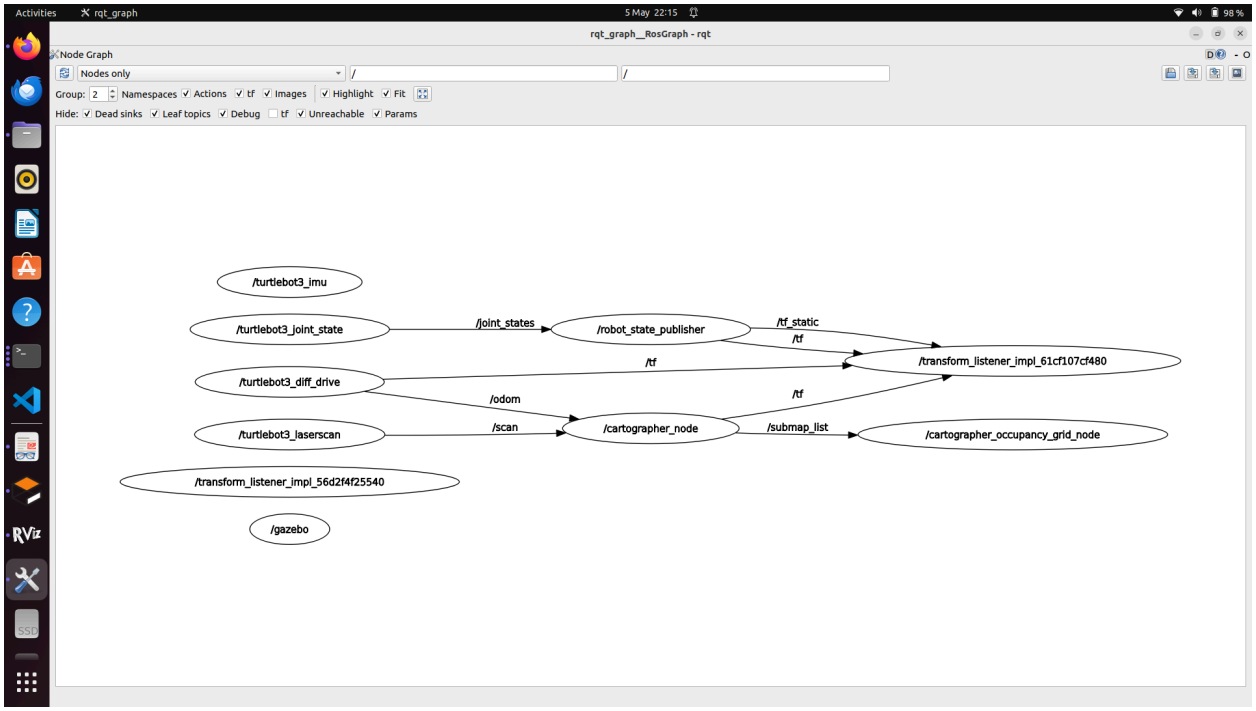
3.1. RViz Visualization & Plugins

The robot was successfully navigated through the simulated environment, building a complete map. The required plugins (TF, Odometry, LaserScan, Path, and Map) were successfully configured and displayed.



3.2. ROS 2 Node Communication Graph

The rqt_graph output below demonstrates the communication network between the Gazebo simulator, Cartographer node, and the robot's joint/state publishers.



4. Observations on Discrepancies

During the teleoperation phase, a noticeable discrepancy between expected and simulated motion was observed. When rapid or consecutive forward commands (w) were issued via the teleop node, the simulated robot would sometimes tilt backward or flip over.

- **Observation:** This occurs because the teleop node increments the target velocity with each key press rather than acting as a standard throttle. In a simulation environment like Gazebo, applying sudden, massive acceleration vectors causes the physics engine to react violently, overcoming the robot's simulated center of gravity. In physical hardware, motor controllers typically smooth out these step inputs to prevent flipping.

5. Custom Node Implementations

5.1. Command Velocity Publisher Task A Python node was created to alternate between publishing a forward velocity (0.2 m/s) and zero velocity every 2 seconds to the /cmd_vel topic.

5.2. Odometry Subscriber Task The message type for the /odom topic was identified as nav_msgs/msg/Odometry. A subscriber node was implemented to extract and print the X and Y coordinate data in real-time.

6. Conclusion

This lab session provided critical hands-on experience with the foundational simulation tools in ROS 2. By successfully integrating Gazebo for physics simulation, Cartographer for SLAM mapping, and RViz for data visualization, I learned how to interpret raw sensor data streams. The primary challenge faced was managing the simulated physics during teleoperation to prevent the robot from flipping, which reinforced the importance of smooth velocity control in mobile robotics. Overall, this lab established the necessary foundation for developing autonomous navigation algorithms in future sessions.